



CPSC 100

Computational Thinking

Intro to Programming

Instructor: Firas Moosvi
Department of Computer Science
University of British Columbia

Agenda

- Revisit Comparing Loops
- Code Explanations
- Code Tracing
- Debugging
- Modulo Operator



Learning Goals

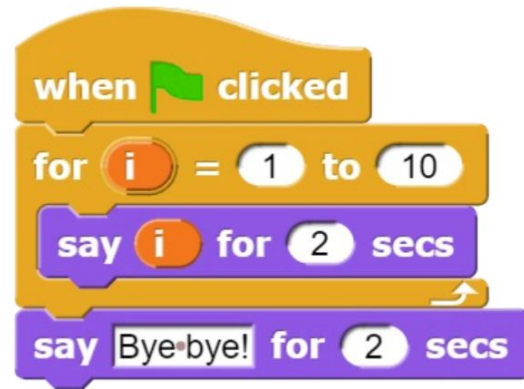
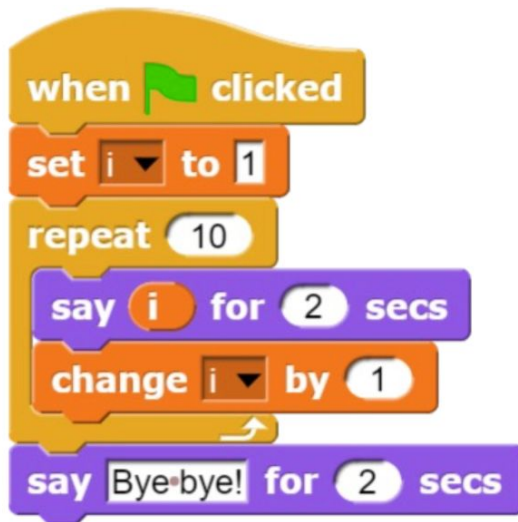
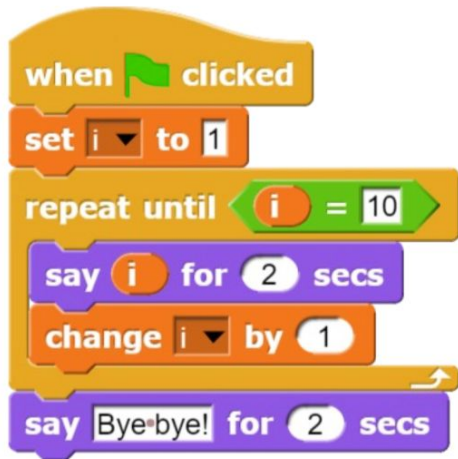
After this today's lecture, you should be able to:

- Identify and understand the differences between **different types of loops** (e.g. repeat, repeat until, for)
- **Trace** through code using sequences of instructions, variables, loops, and conditional statements in short programs
- **Describe in English** what a block of Snap! code does.
- **Evaluate** if a given snap code block correctly implements an algorithm.

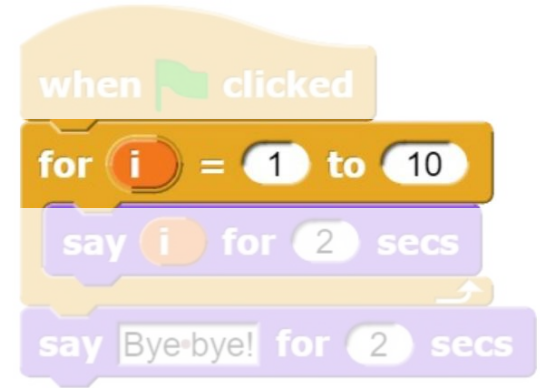
Course Admin

Comparing Loops

What's the difference between these loops?



What's the difference between these loops?



Code Explanation



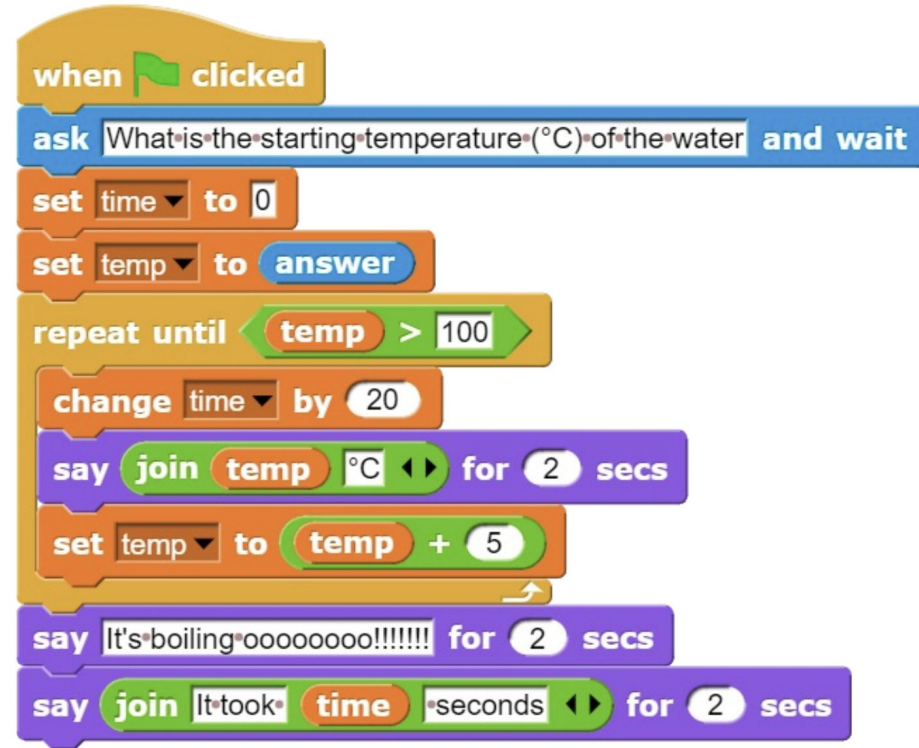
What does this code block do?

Describe the code in terms of input and output and what is being done.

The block consumes....

It does...

It produces/displays/reports...

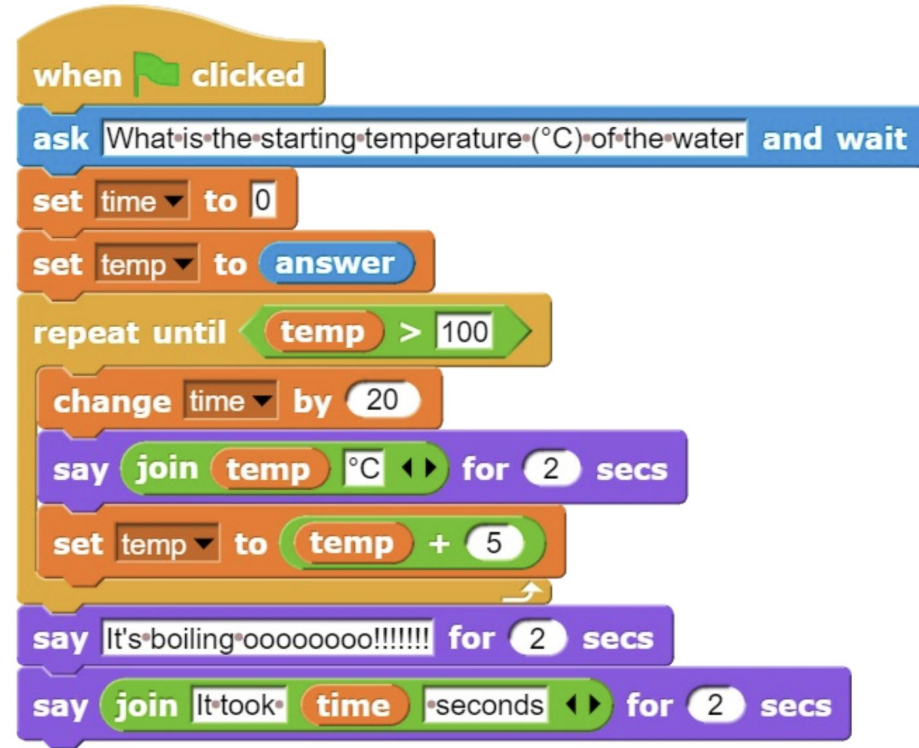




What is the purpose of this block?

The block consumes the starting temperature for water
It increases the temperature and time

It produces the time it takes to get the water from the starting temperature to 100 degree Celsius

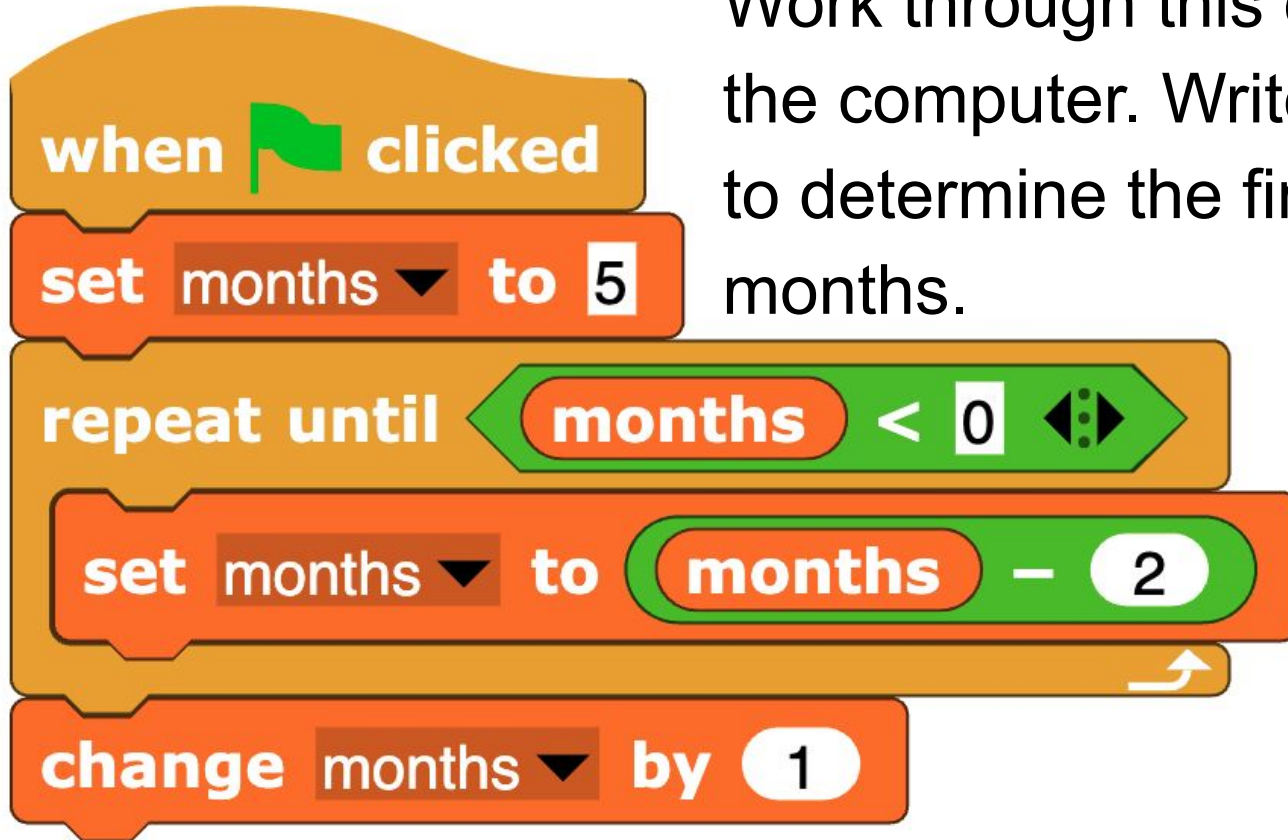


Code Tracing Demo



Activity - Step through this code

Work through this code as if you're the computer. Write down each step to determine the final value of months.

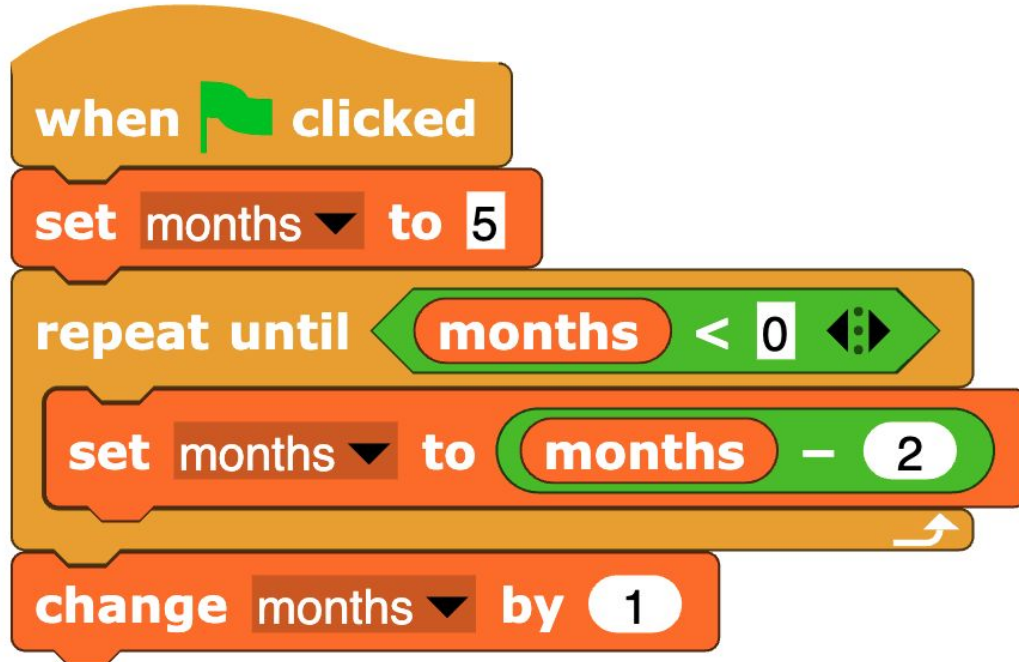




Activity - Step through this code

Step 1: Write down the variables you want to track.

months

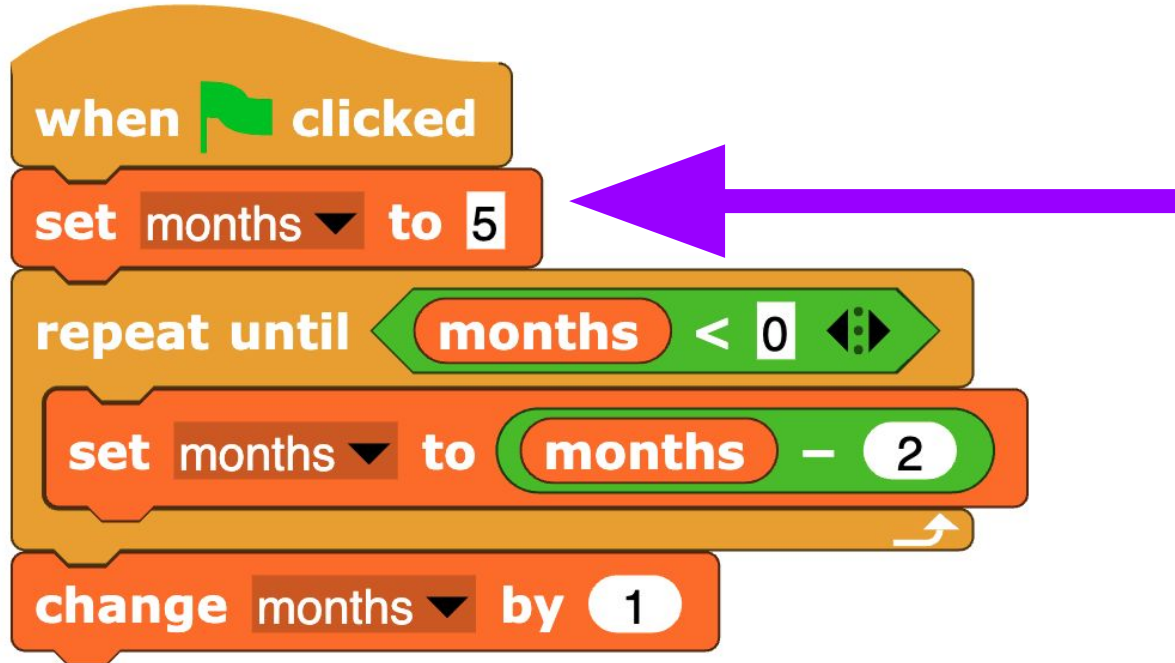
☐



Activity - Step through this code

Step 2.0: Write down the value of each variable before entering the loop/repeat block.

months 5

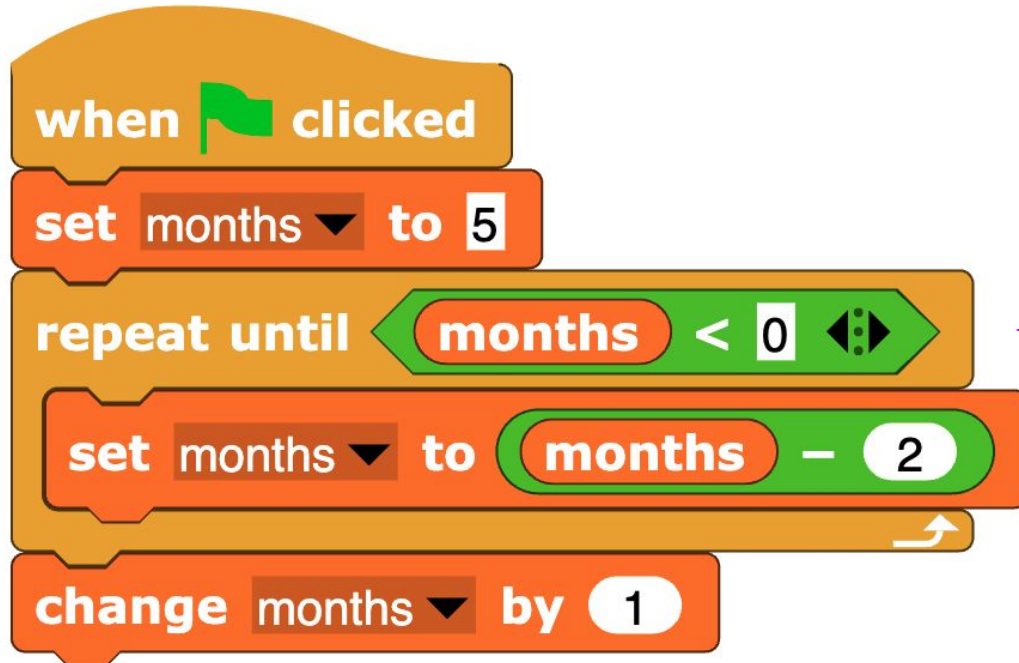




Activity - Step through this code

Step 2.1: Go through each step of the loop, keep track of the variable and its value.

months 5



Is the condition true?

No → Execute code inside the loop.

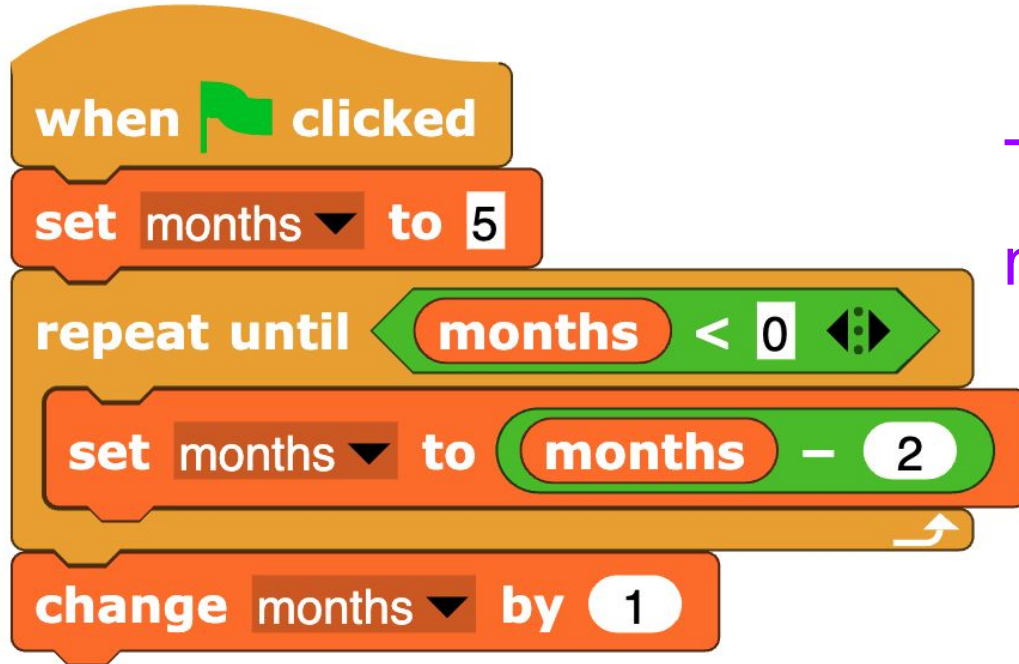
Yes → Exit the loop, execute the first line after the loop.



Activity - Step through this code

Step 2.2: Go through each step of the loop, keep track of the variable and its value.

months 5



The value stored inside the months variable changes:

$$5 - 2 = 3$$

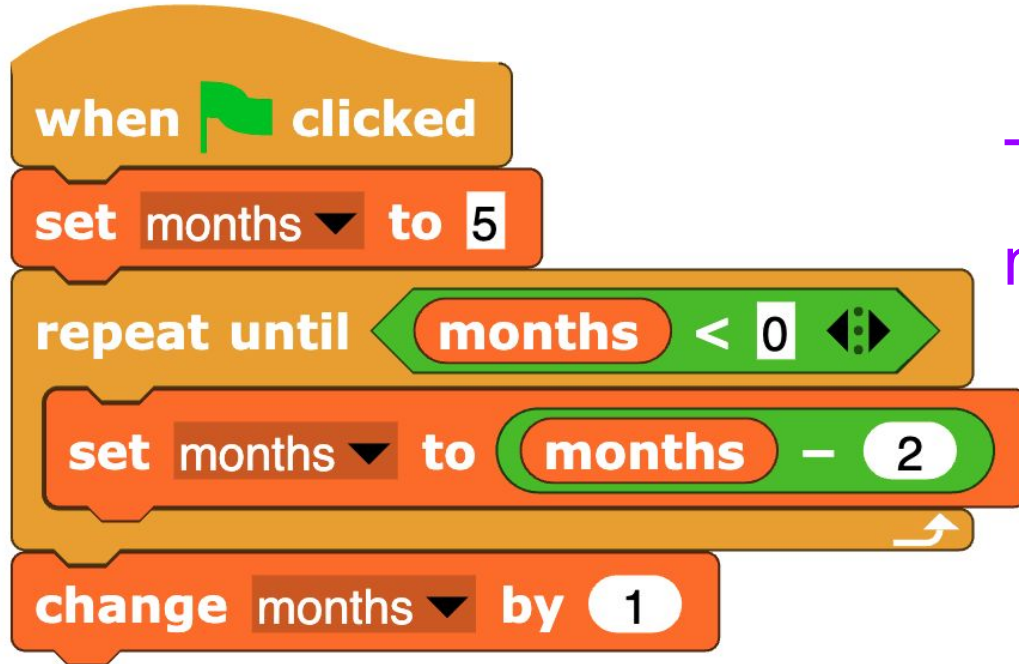




Activity - Step through this code

Step 2.2: Go through each step of the loop, keep track of the variable and its value.

months **3**



The value stored inside the months variable changes:

$$5 - 2 = 3$$

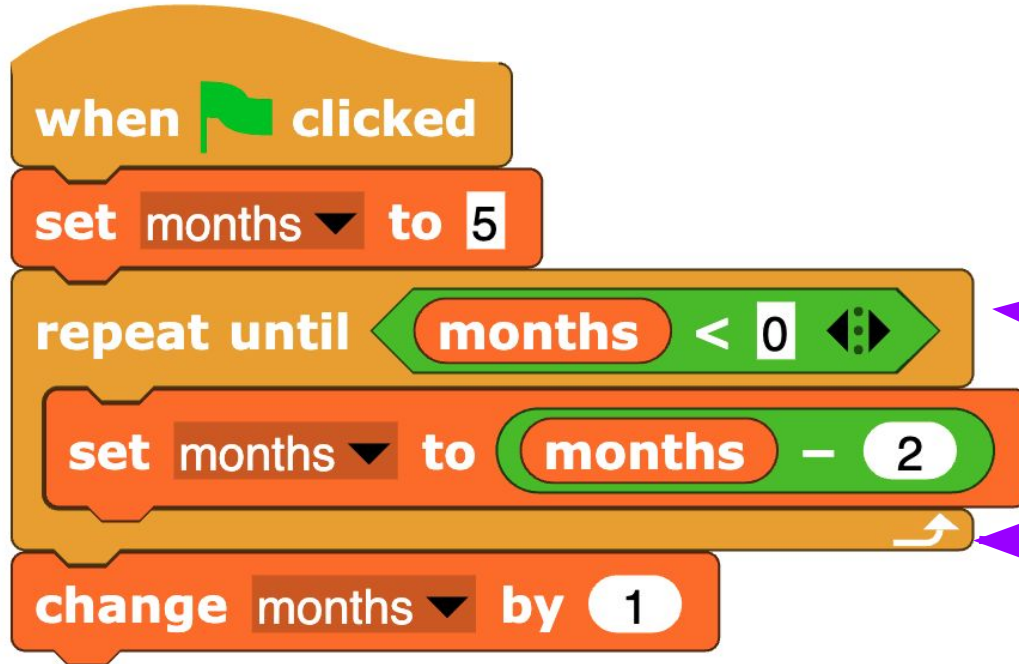




Activity - Step through this code

Step 2.3: Go through each step of the loop, keep track of the variable and its value.

months 3



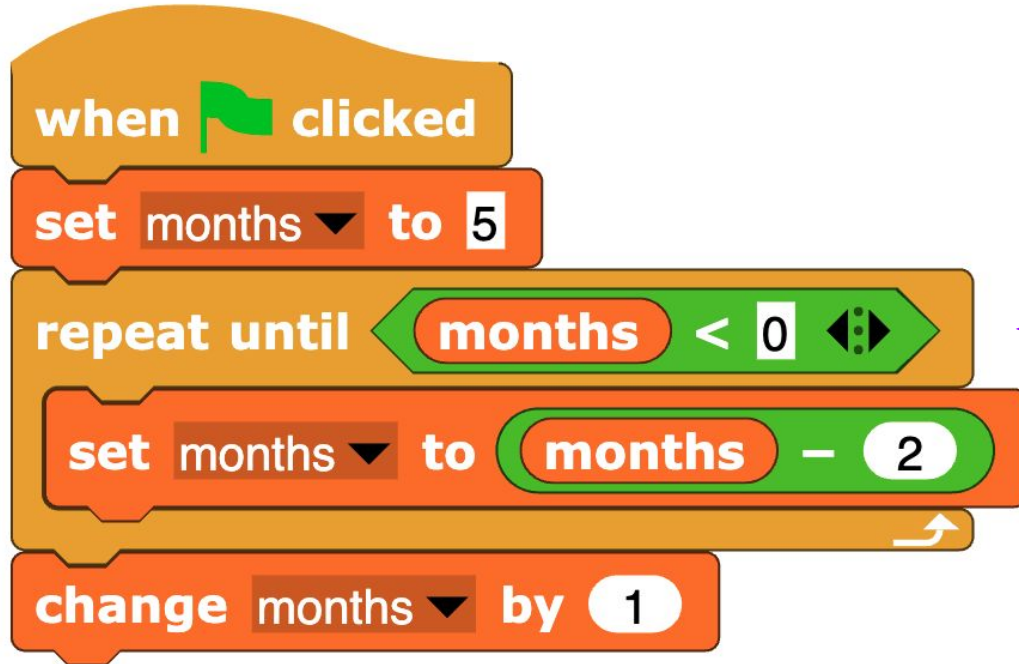
No more code inside the loop. We head back up to the start of the loop.



Activity - Step through this code

Step 2.4: Go through each step of the loop, keep track of the variable and its value.

months 3



Is the condition true?

No → Execute code inside the loop.

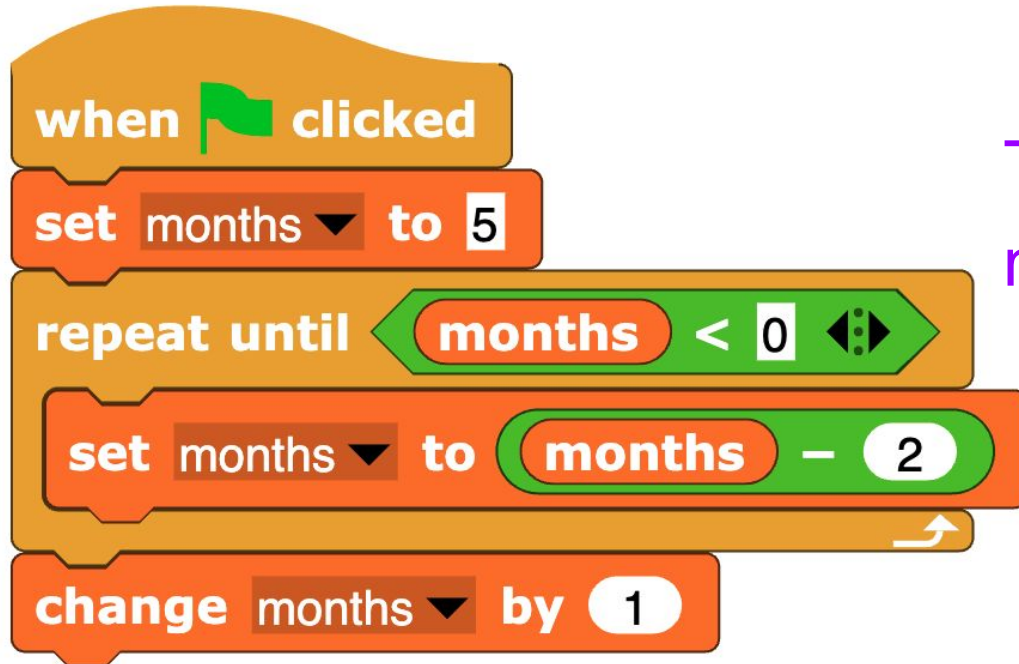
Yes → Exit the loop, execute the first line after the loop.



Activity - Step through this code

Step 2.5: Go through each step of the loop, keep track of the variable and its value.

months 3



The value stored inside the months variable changes:

$$3 - 2 = 1$$

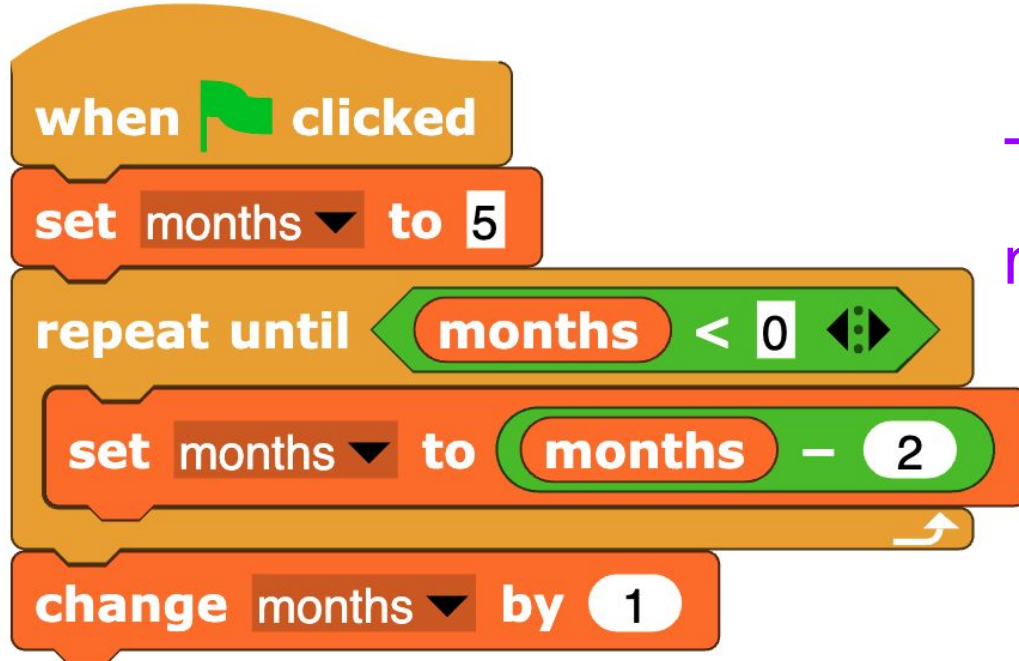




Activity - Step through this code

Step 2.5: Go through each step of the loop, keep track of the variable and its value.

months 1



The value stored inside the months variable changes:

$$3 - 2 = 1$$

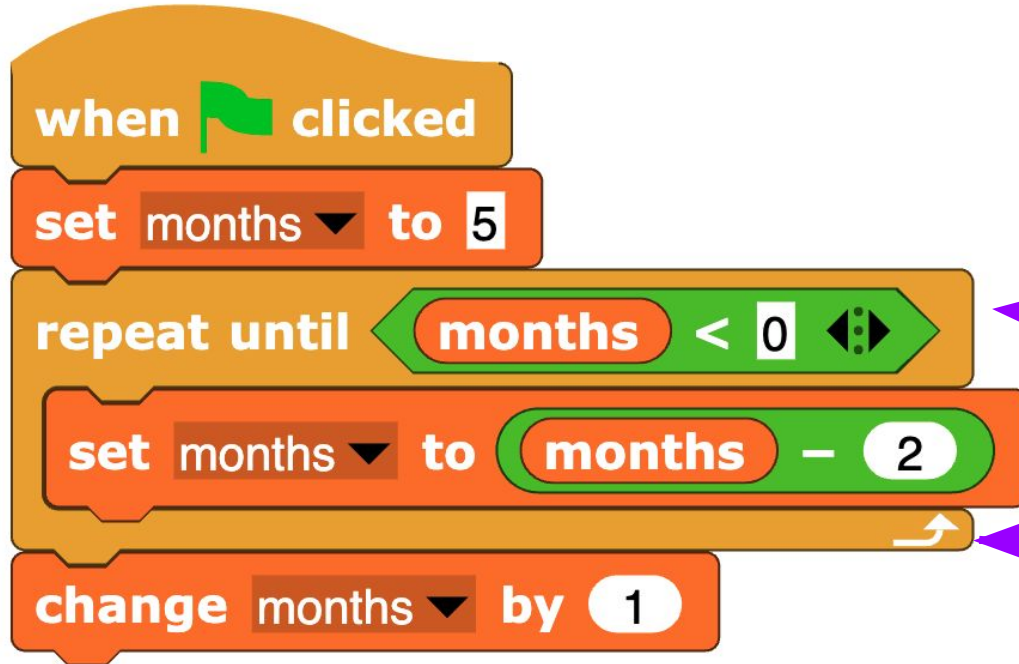




Activity - Step through this code

Step 2.6: Go through each step of the loop, keep track of the variable and its value.

months 1



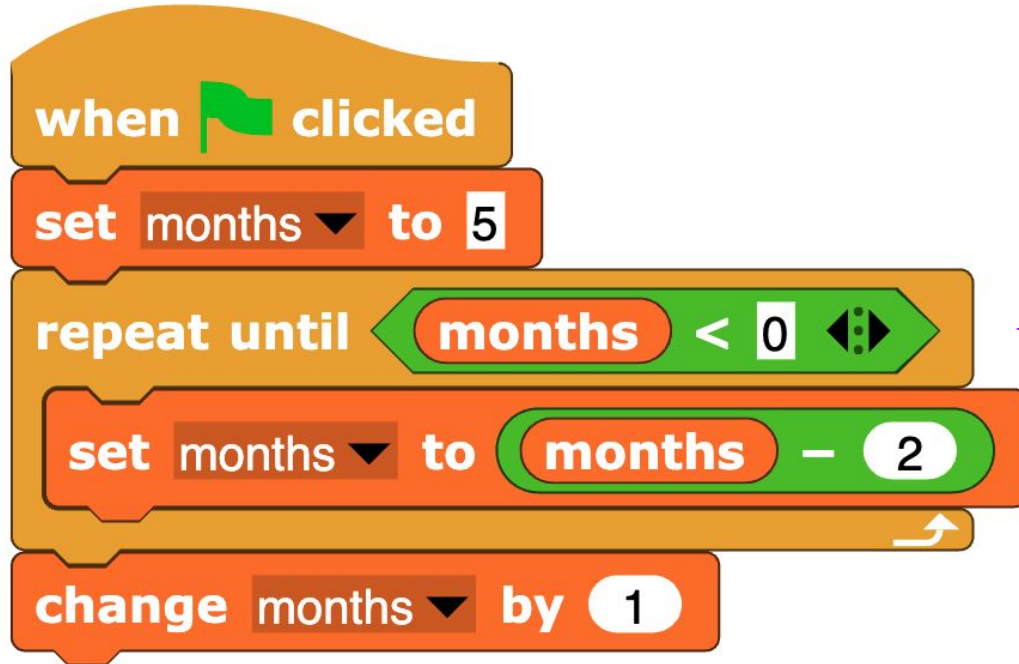
No more code inside the loop. We head back up to the start of the loop.



Activity - Step through this code

Step 2.7: Go through each step of the loop, keep track of the variable and its value.

months 1



Is the condition true?

No → Execute code inside the loop.

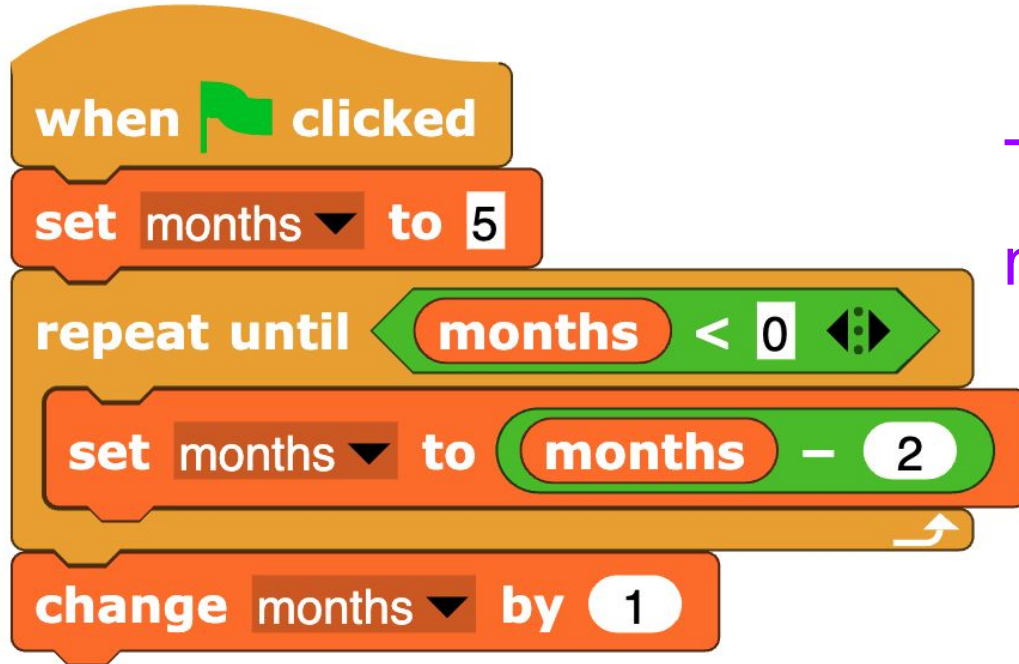
Yes → Exit the loop, execute the first line after the loop.



Activity - Step through this code

Step 2.8: Go through each step of the loop, keep track of the variable and its value.

months 1



The value stored inside the months variable changes:

$$1 - 2 = -1$$

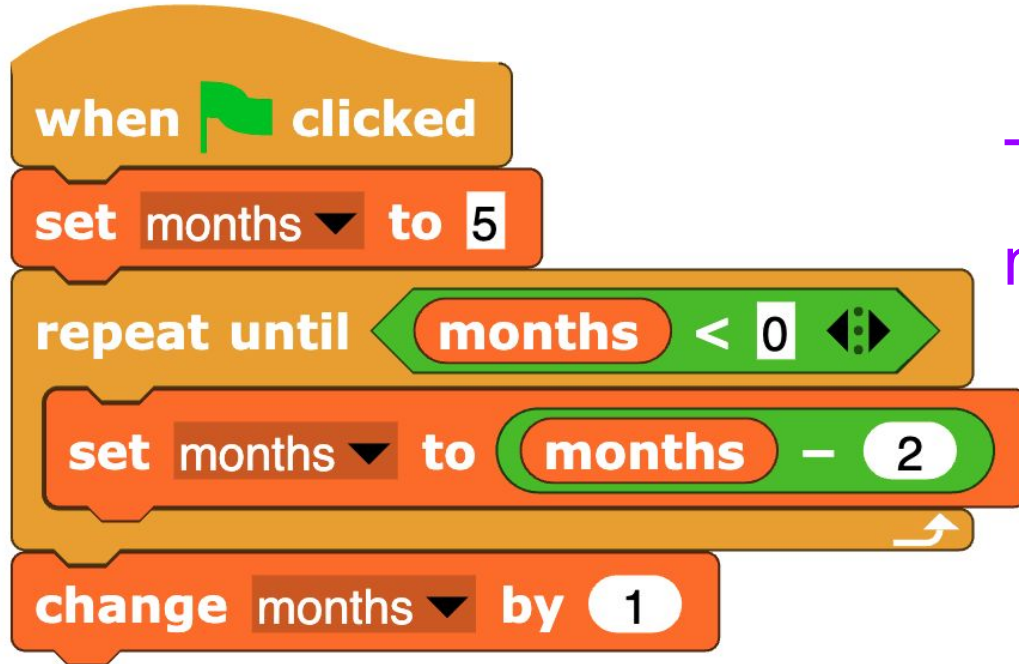




Activity - Step through this code

Step 2.8: Go through each step of the loop, keep track of the variable and its value.

months -1



The value stored inside the months variable changes:

$$1 - 2 = -1$$

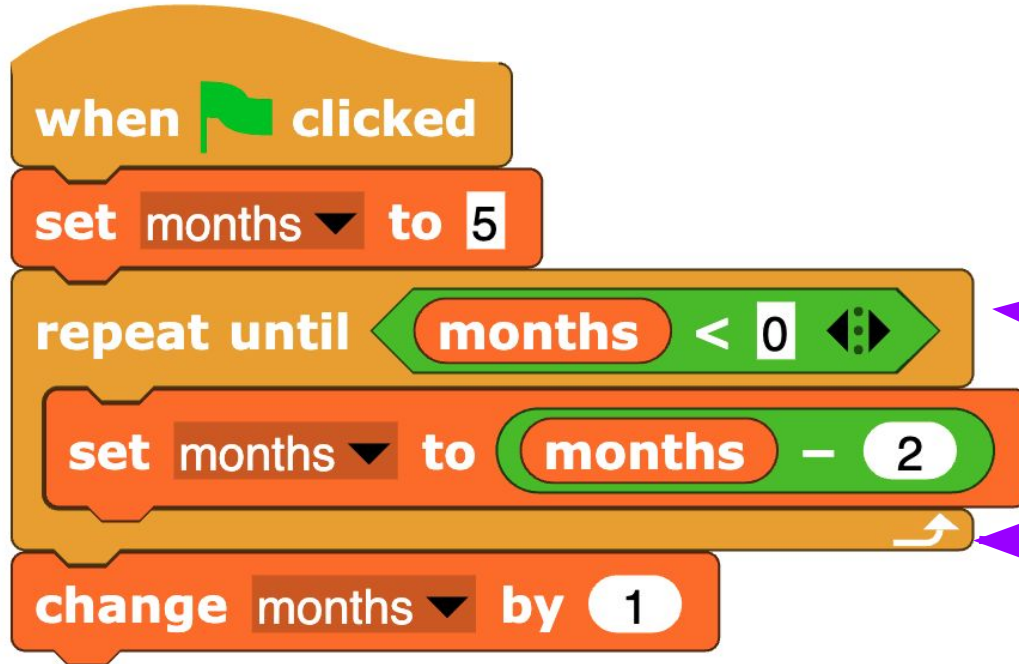




Activity - Step through this code

Step 2.9: Go through each step of the loop, keep track of the variable and its value.

months -1



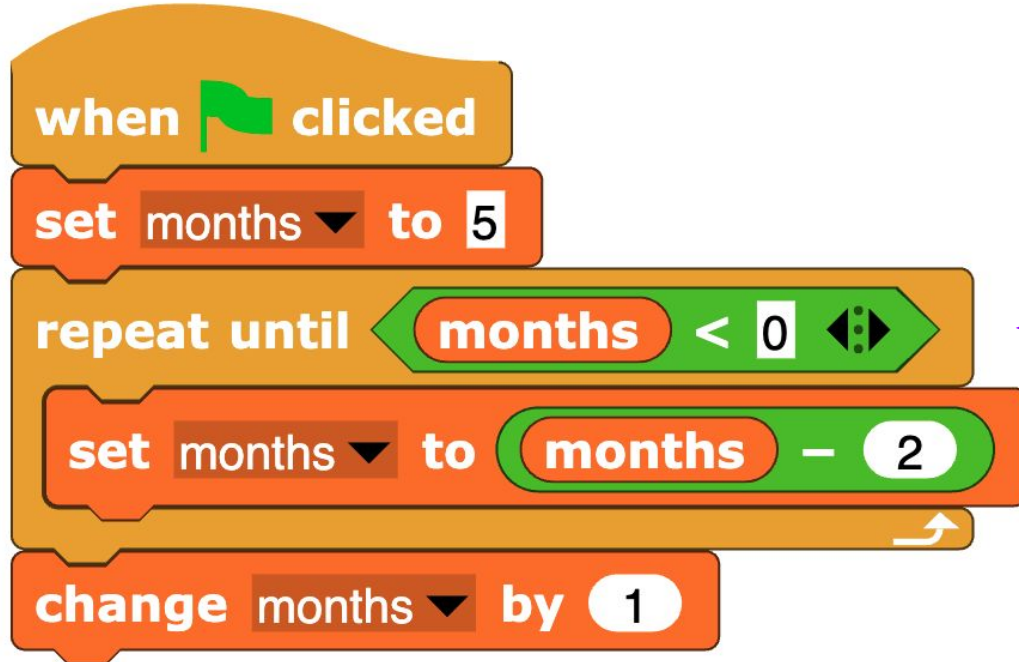
No more code inside the loop. We head back up to the start of the loop.



Activity - Step through this code

Step 3.0: Write down the value of each variable after exiting the loop/repeat block.

months -1



Is the condition true?

No → Execute code inside the loop.

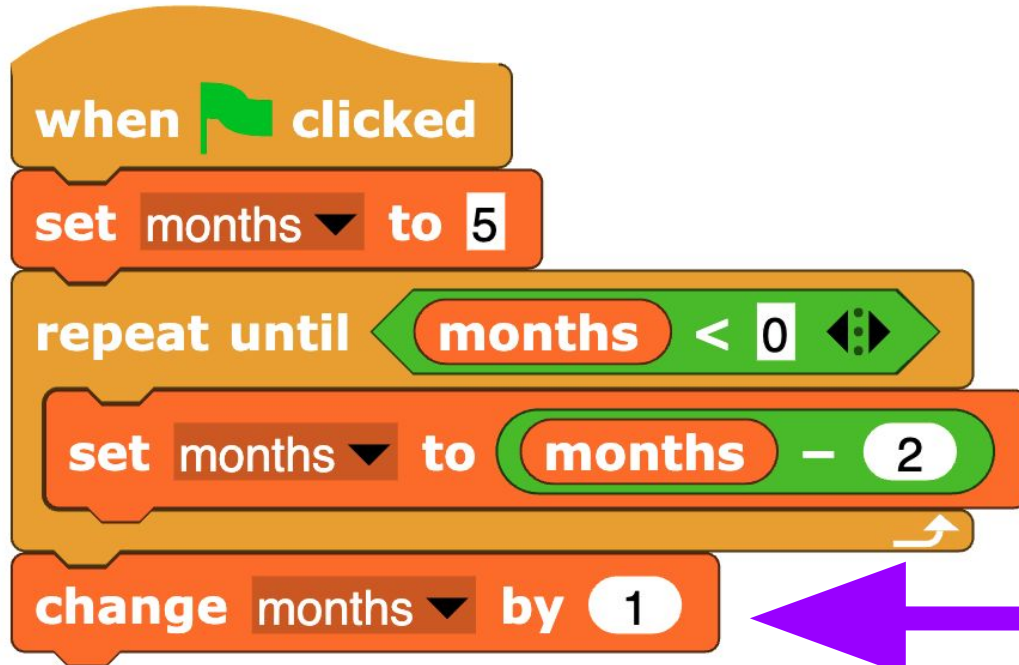
Yes → Exit the loop, execute the first line after the loop.



Activity - Step through this code

Step 3.1: Go through any remaining code, keep track of the variable and its value.

months 0



The value stored inside the months variable changes:

$$-1 + 1 = 0$$

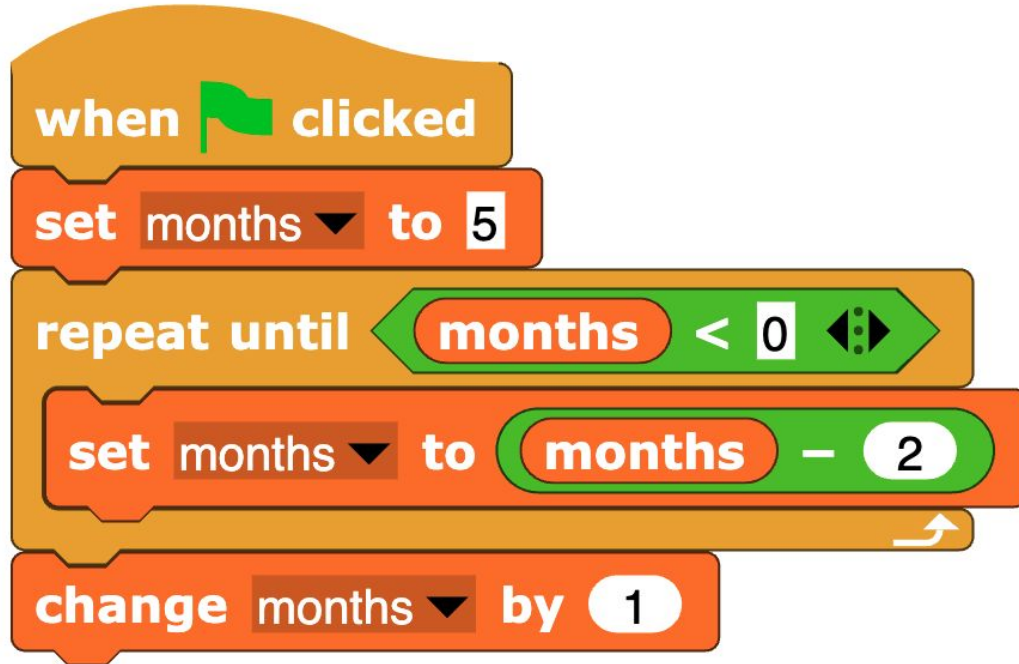




Activity - Step through this code

Step 4: Celebrate! You have solved the question

months 0



Done!

Debugging

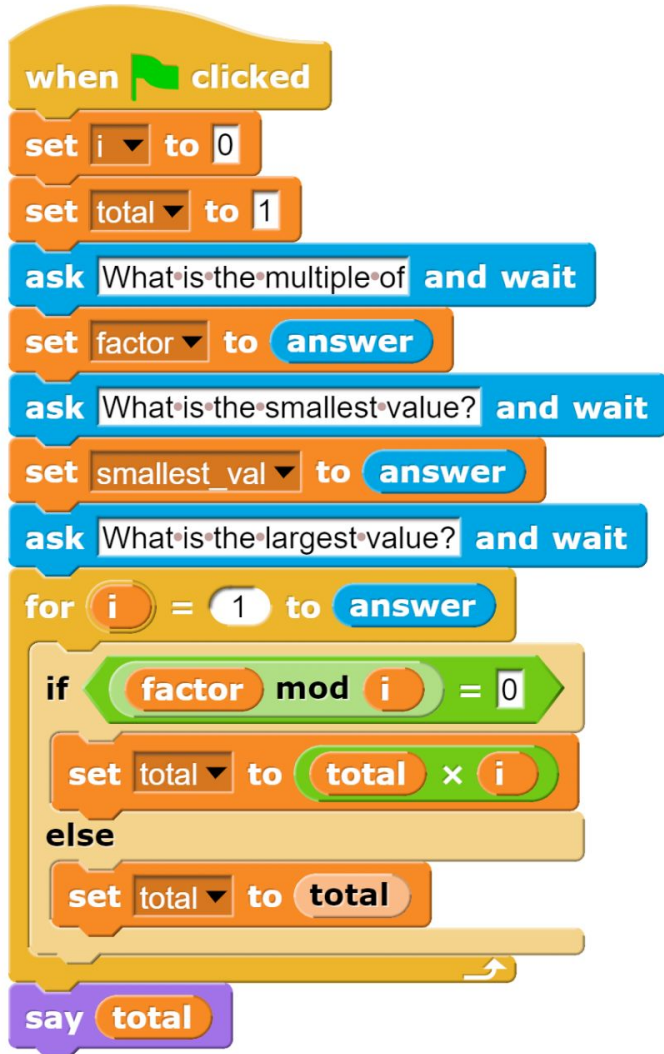
92

9/9

0800 Antan started
 1000 " stopped - antan ✓
 { 1.2700 - 9.037 847 025
 9.037 846 795 const
 13" uc (032) MP - MC 1.582677000
 2.130476415 (033) 4.615925059(-2)
 (033) PRO 2 2.130476415
 const 2.130676415
 Relays 6-2 in 033 failed special speed test
 in relay " 11.000 test.
 Relays changed
 1700 Started Cosine Tape (Sine check)
 1525 Started Multi Adder Test.
 1545  Relay #70 Panel F
 (moth) in relay.
 First actual case of bug being found.
 1630 Antan started.
 1700 closed down.



Activity



This code block is supposed to find the product between two positive integers (not inclusive)

Example, if the user inputs:

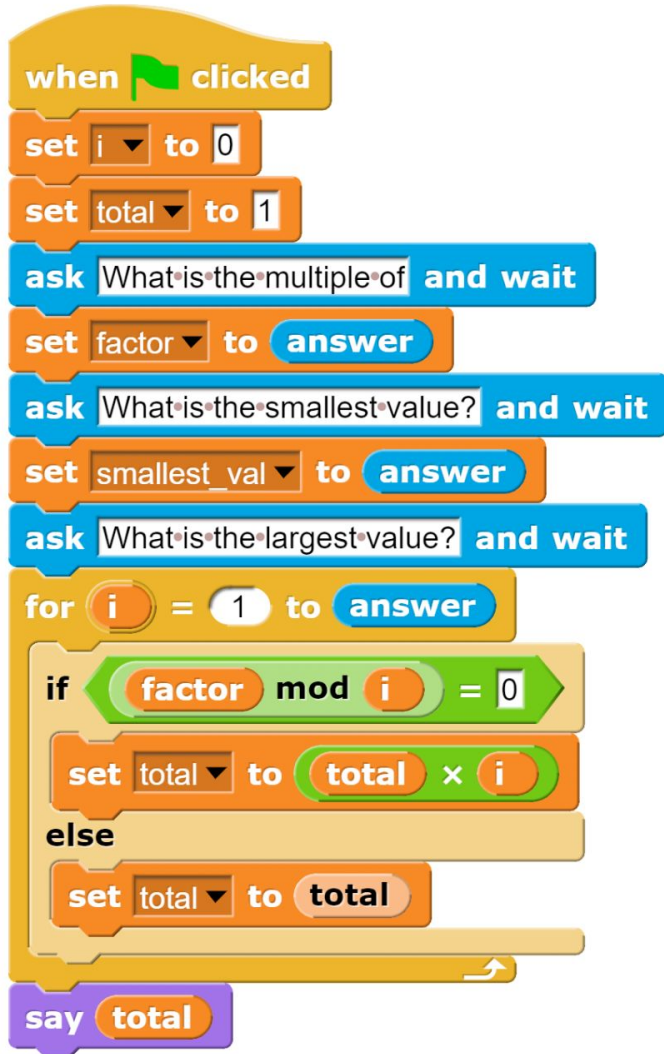
What is the multiple of? 2

What is the smallest value? 1

What is the largest value? 10

The result should be:

$$2 \times 4 \times 6 \times 8 = 384$$



Review the code block and identify any bug(s).

1. Clearly highlight the problematic code [bug(s)]
2. Explain in plain English what needs to be changed so the code works properly

Modulo Operator



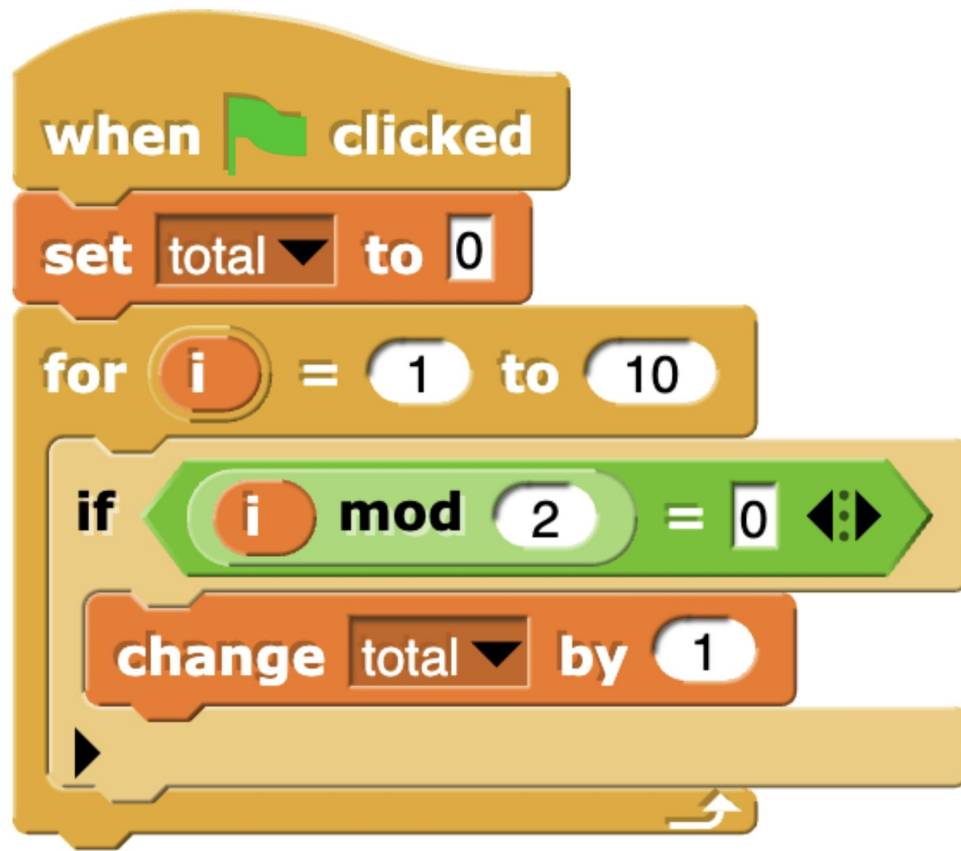
Learning Goals

After this today's lecture, you should be able to:

- Understand and explain the modulo (mod) operator.
- Apply mod operator in Snap! programming
- Understand the history & importance of debugging in programming
- Identify any bugs associated with a given code block
- Explain in plain English what needs to be changed to resolve bugs
- *Bonus: understand AM/PM acronym in the clock system*

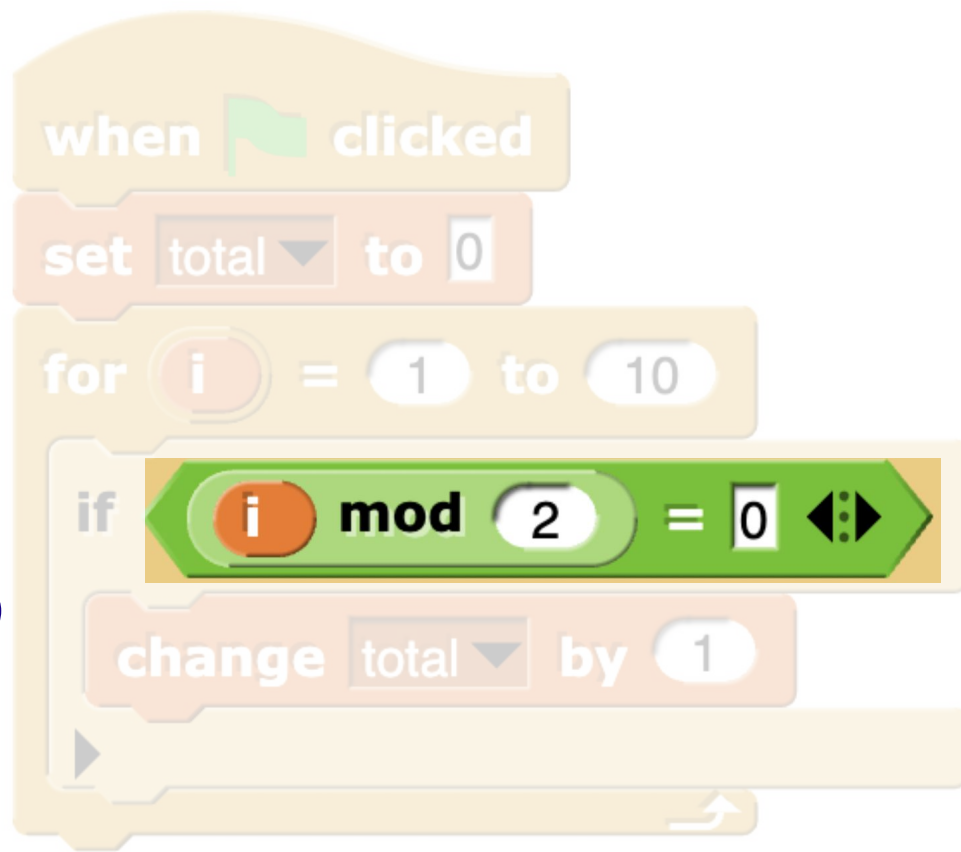
Q: What is the value of **total** when this code block is run?

What does this code block do?



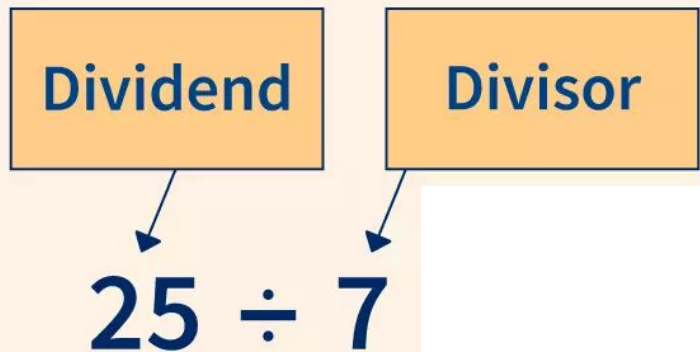
"Mod" Operator

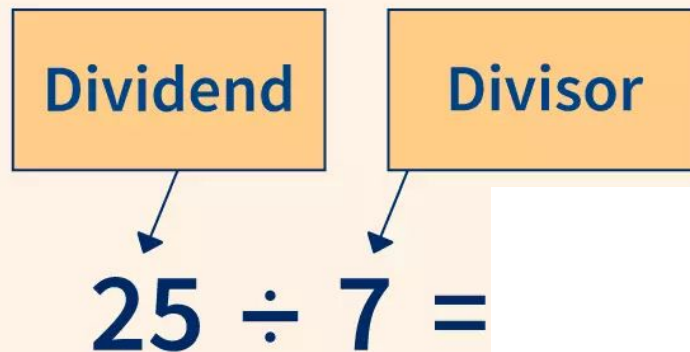
Short for *modulo*





$$25 \div 7$$





Dividend

Divisor

$$25 \div 7 = 3 \text{ remainder } 4$$

Dividend

Divisor

$$25 \div 7 = 3 \text{ remainder } 4$$

$$25 \% 7 = 4$$

Modulo Operator: return the remainder of a division



Examples

- Clock System (AM/PM)
 - Before noon: **A**nte **M**eridiem (AM) → 12:01am to 11:59am
 - After noon: **P**ost **M**eridiem (PM) → 12:00pm to 11:59pm



Examples

- Clock System (AM/PM)
 - Before noon: **A**nte **M**eridiem (AM) → 12:01am to 11:59am
 - After noon: **P**ost **M**eridiem (PM) → 12:00pm to 11:59pm
- Clock System (military)
 - 24 hour system → after 12: we keep on counting (13, 14..)



Examples

- Clock System (AM/PM)
 - Before noon: **A**nte **M**eridiem (AM) → 12:01am to 11:59am
 - After noon: **P**ost **M**eridiem (PM) → 12:00pm to 11:59pm
- Clock System (military)
 - 24 hour system → after 12: we keep on counting (13, 14..)
- To convert between these two, we use a **mod operator**!
 - Our class starts at 15:00 → 3pm
 - $15 \bmod 12 = 3$ (*since dividing 15 by 12, the remainder is 3*)



More Examples

- $5 \bmod 2 = 1$ (the closest divisor is [2], $2 \times 2 = \mathbf{4}$, the remainder is 1)
- $9 \bmod 3 = 0$ (since 9 is exactly divisible by 3 with **no** remainder)
- $17 \bmod 5 = 2$ (the closest divisor is [3], $5 \times 3 = \mathbf{15}$, the remainder is 2)



That's it!

That's all the
programming
basics you need to
know *...(for now).*

Programming is a LOT
easier to learn by
doing than by
watching!

Wrap up

Take-Home Practice

Q: There's no $\leq$ block in Snap! Suppose we wanted to build one. Which of the following Boolean expressions is equivalent to the expression  ?



iClicker

A



B



C



D



Q: There's no $\leq$ block in Snap! Suppose we wanted to build one. Which of the following Boolean expressions is equivalent to the expression  ?



iClicker

A



B



C



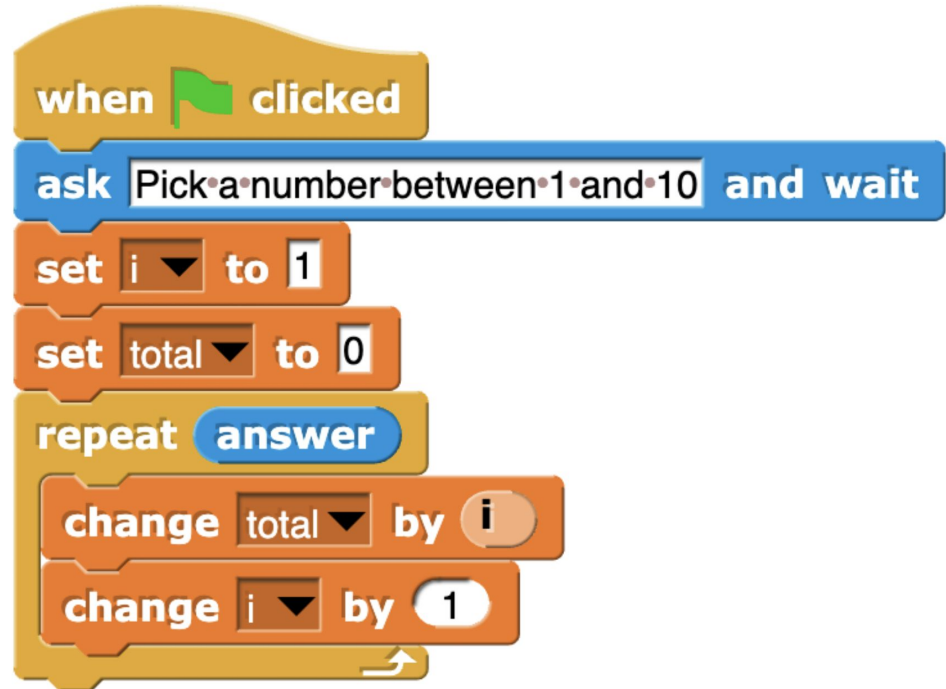
D



Q: What is the value in **total when the code is run, assuming **user input = 3**?**



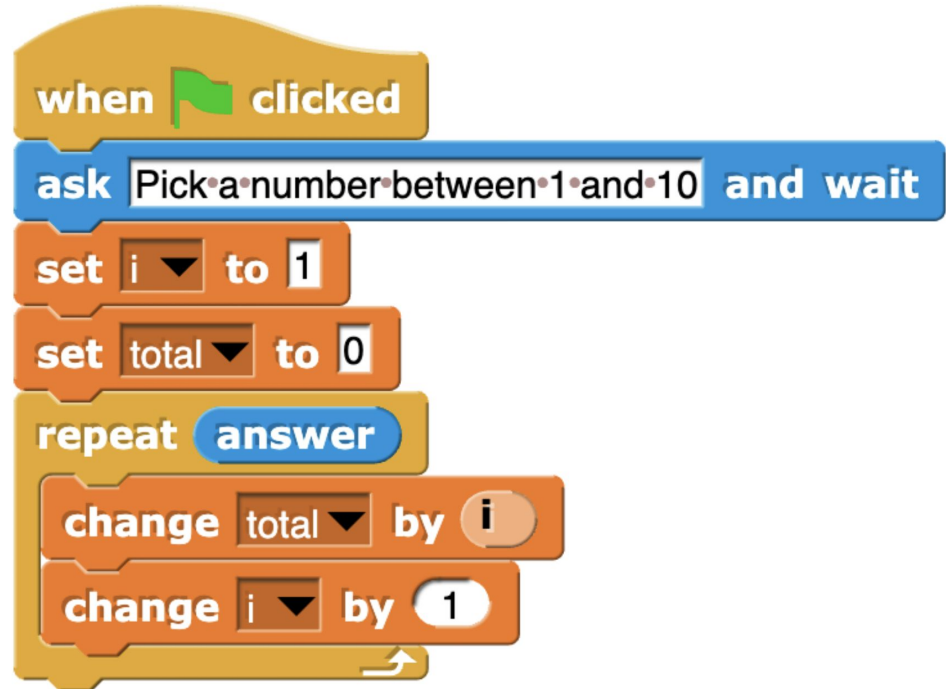
- A. 2
- B. 3
- C. 4
- D. 6
- E. 10



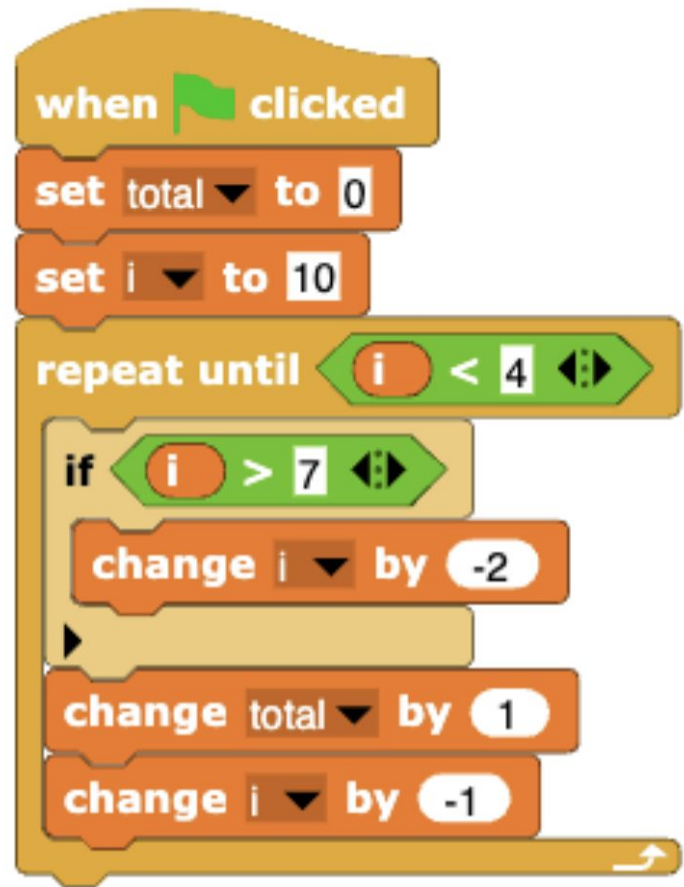
Q: What is the value in **total when the code is run, assuming **user input = 3**?**



- A. 2
- B. 3
- C. 4
- D. 6
- E. 10

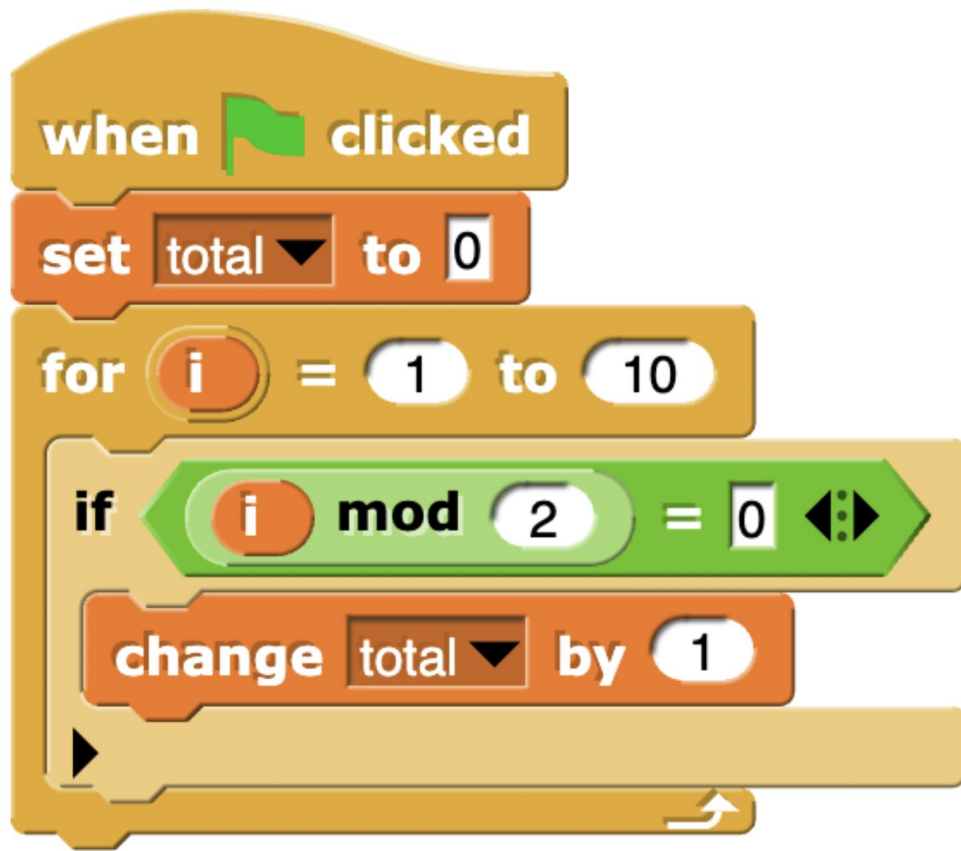


Q: What is the value of **total** and **i** when this code block is run?



Q: What is the value of **total when this code block is run?**

What does this code block do?



Q: What will the following expression evaluate to based on the given values?



a < b and not c > b and d = a

- A. True
- B. False

```

set a to 5
set b to 3
set c to 3
set d to 5
    
```

Q: What will the following expression evaluate to based on the given values?



a < b and not c > b and d = a

A. True

B. False

```

set a to 5
set b to 3
set c to 3
set d to 5
    
```

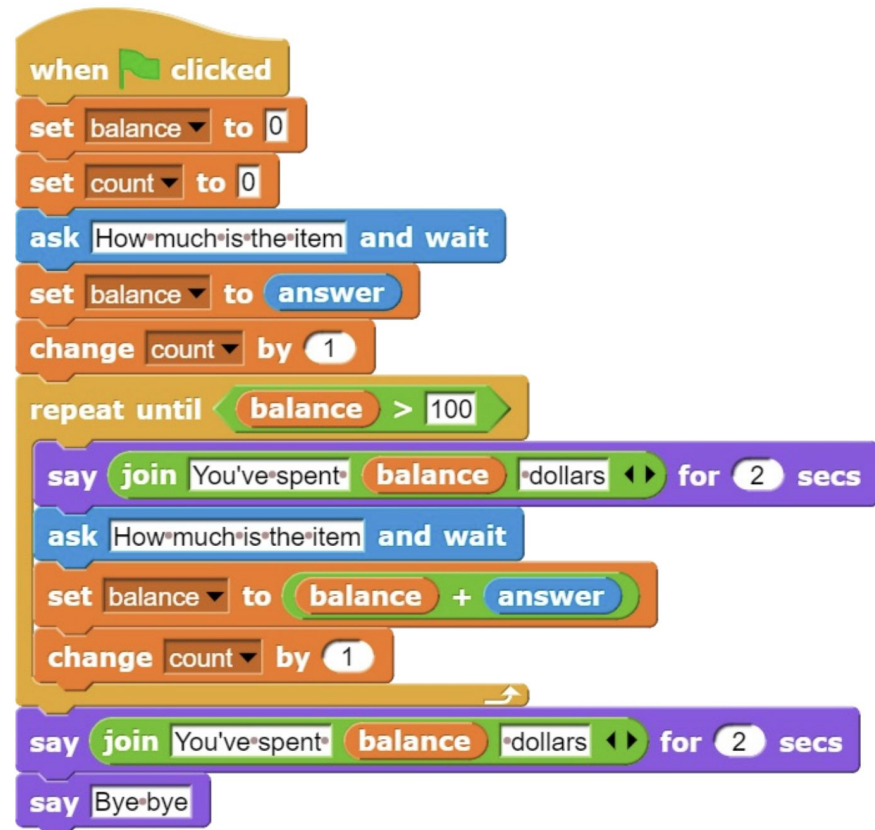
What does this code block do?

Describe the code in terms of input and output and what is being done.

The block consumes....

It does...

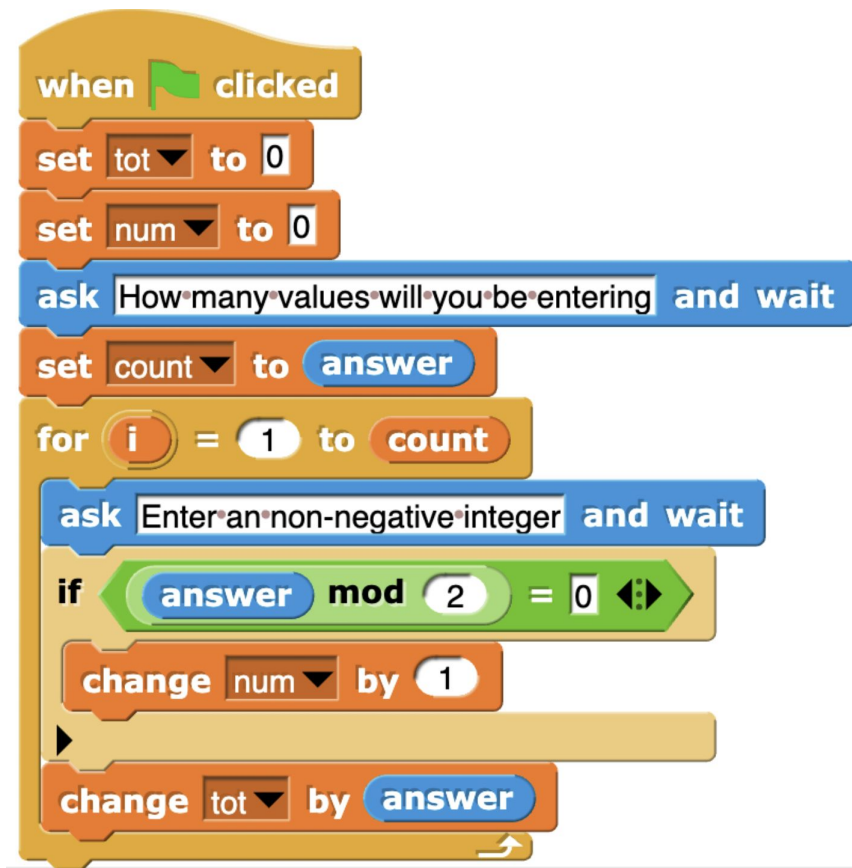
It produces/displays/reports...



What does this code block do?

What is the value of **tot**, **num** and **count** when the block is run? *[for practice, you can assume a number between 1-6]*

What does this code block do?



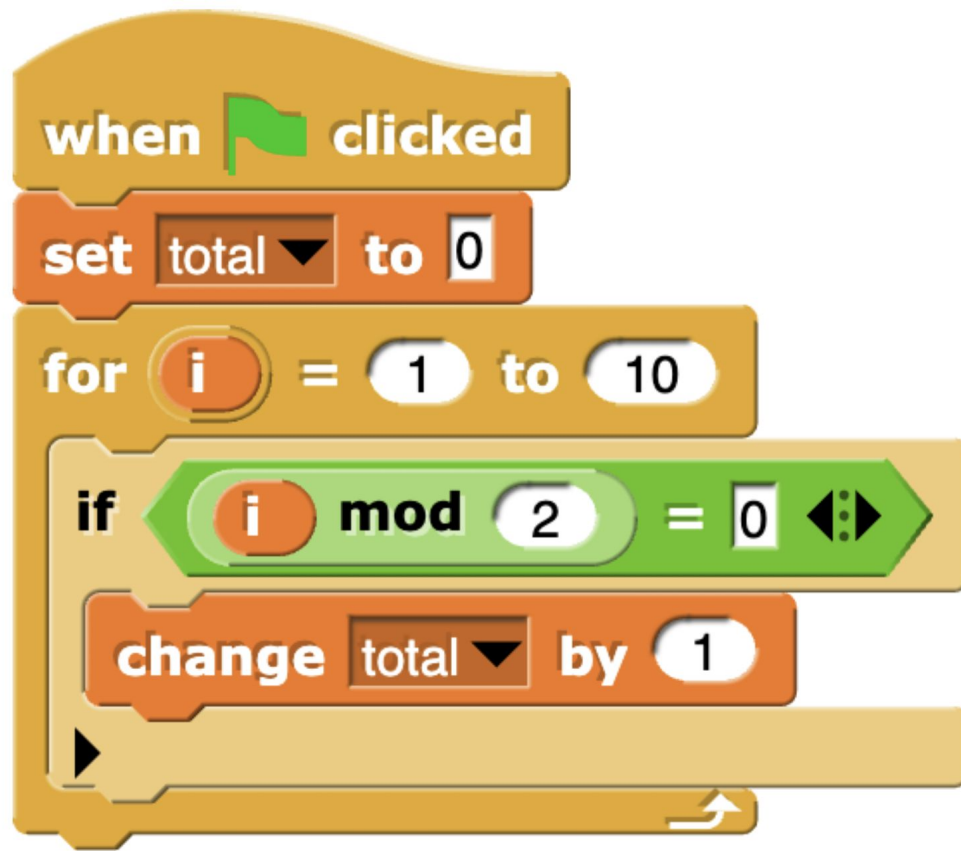
Q: What will the following expression report?



- A. True
- B. False

Q: What is the value of **total when this code block is run?**

What does this code block do?



More Examples

- $5 \bmod 2 = 1$ (the closest divisor is [2], $2 \times 2 = \mathbf{4}$, the remainder is 1)
- $9 \bmod 3 = 0$ (since 9 is exactly divisible by 3 with **no** remainder)
- $17 \bmod 5 = 2$ (the closest divisor is [3], $5 \times 3 = \mathbf{15}$, the remainder is 2)
- $25 \bmod 3 = 1$ (the closest divisor is [8], $3 \times 8 = \mathbf{24}$, the remainder is 1)
- $44 \bmod 10 = 4$ (the closest divisor is [4], $10 \times 4 = \mathbf{40}$, the remainder is 4)
- $53 \bmod 6 = 5$ (the closest divisor is [8], $6 \times 8 = \mathbf{48}$, the remainder is 5)
- $72 \bmod 8 = 0$ (since 72 is exactly divisible by 8 with **no** remainder)

$$2 \bmod 2 =$$

$$17 \bmod 3 =$$

$$40 \bmod 9 =$$

$$1 \bmod 2 =$$

$$61 \bmod 8 =$$

$$37 \bmod 7 =$$

$$153 \bmod 4 =$$

2

mod

2

= 0

17

mod

3

= 2

40

mod

9

= 4

1

mod

2

= 1

61

mod

8

= 5

37

mod

7

= 2

153

mod

4

= 1



Programming Context:

- This operator is helpful in programming to check **if a number is even or odd, looping through a range of values, and creating patterns.**



Programming Context:

- This operator is helpful in programming to check **if a number is even or odd, looping through a range of values, and creating patterns.**
- An even number will have a remainder of 0 when divided by 2, while an odd number will have a remainder of 1
 - $7 \bmod 2 = 1$ (Odd)
 - $12 \bmod 2 = 0$ (Even)



2 mod 2

17 mod 2

40 mod 2

1 mod 2

61 mod 2

37 mod 2

153 mod 2

**What do
these
Arithmetic
Operators
evaluate to?
Odd or even?**

2

mod

2

= 0 (even)

17

mod

2

40

mod

2

1

mod

2

61

mod

2

37

mod

2

153

mod

2

2

mod

2

= 0 (even)

17

mod

2

= 1 (odd)

40

mod

2

1

mod

2

61

mod

2

37

mod

2

153

mod

2

2

mod

2

= 0 (even)

17

mod

2

= 1 (odd)

40

mod

2

= 0 (even)

1

mod

2

61

mod

2

37

mod

2

153

mod

2

2

mod

2

= 0 (even)

17

mod

2

= 1 (odd)

40

mod

2

= 0 (even)

1

mod

2

= 1 (odd)

61

mod

2

37

mod

2

153

mod

2

2

mod

2

= 0 (even)

17

mod

2

= 1 (odd)

40

mod

2

= 0 (even)

1

mod

2

= 1 (odd)

61

mod

2

= 1 (odd)

37

mod

2

153

mod

2

2

mod

2

= 0 (even)

17

mod

2

= 1 (odd)

40

mod

2

= 0 (even)

1

mod

2

= 1 (odd)

61

mod

2

= 1 (odd)

37

mod

2

= 1 (odd)

153

mod

2

2	mod	2	= 0 (even)
17	mod	2	= 1 (odd)
40	mod	2	= 0 (even)
1	mod	2	= 1 (odd)
61	mod	2	= 1 (odd)
37	mod	2	= 1 (odd)
153	mod	2	= 1 (odd)

